

## Elvárások a programmal szemben

- A megoldáshoz semmilyen segédeszköz nem használható, kivéve a C referenciát.
- A program végleges verziójának működőképesnek kell lennie. Forduljon és fusson!
  - **A nem forduló kód 0 pontot ér!**
- Ne használj globális változókat! Csak a makrók megengedettek!
- Törekedj a szép, áttekinthető kódolásra, használj indentálást, kerülj a kódismétlést!
- Kommunikáld a program! Legyen egyértelmű a felhasználó számára, hogy mit vár a program, illetve pontosan mi történik!
- Logikusan tagold a megoldást (használd függvényeket, külön fordítási egységeket)!
- Ne foglald szükségtelenül memóriát, és kerülj a memóriaszivárgást!
- Kerüld a nem definiált viselkedést okozó utasításokat!
- Ahol nincs pontosan leírva egy feladatrészt, annak egyedi megvalósítását rád bízunk.
- A struktúrák paraméterként átadásánál gondold át, hol érdemes mutatót használni!
- **A végleges programban a be nem tartott elvárások pontlevonással járnak!**

## Rajztábla

A feladat egy rajztábla megvalósítása C nyelven. A zárthelyi két fő részre bontható: Az első felében a „pixeleket” megjelenítő rajztáblát fogod elkészíteni, melyet meg is jelenítesz az alapértelmezett kimeneten (terminál).

A feladata második felében erre a táblára fogsz egyszerű formákat (téglalapok) rajzolni.



A feladat összesen **60** pontot ér.

Legalább **30** pontot kell gyűjteni a tárgy sikeres teljesítéséhez.

A megoldásra **180 perc** áll rendelkezésre.

Beadáskor csak a **.c**, **.h** és **.txt** fájlokat kell becsomagolva feltölteni a következő formában: **<neptun\_kód>.zip** (Az utoljára feltöltött megoldást pontozzuk.)

### colors.h

A feladat megoldásához biztosítunk egy kód fájlt, aminek a segítségével meg lehet változtatni a terminálra kiírt szöveg háttérszínét. A korábban említett pixelek igazából különböző háttérszínű szóközi karakterek lesznek. Használd fel az előre megírt felsorolási típust, valamint alprogramokat a saját kódodban, a pixelek tárolásához, kirajzolásához, terminál törléséhez.

*Megjegyzés: A programod implementációjától függően a colors.h fájl használatával kapcsolatban generálhat figyelmeztetéseket a fordító. Ezeket nyugodtan hagyd figyelmen kívül, nem számítanak hibának!*

### Adatstruktúra ( 16 pont )

Hozd létre a **canvas** struktúrát, amely a rajzvásznat fogja reprezentálni. Egy kétdimenziós pixel mátrixot kell elképzelned, ahol minden egyes pixel, a **colors.h**-ban megadott színek valamelyikét veheti fel. A struktúra tárolja a rajzvásznon dimenzióit: a szélességét és a magasságát. Mivel nem tudjuk előre a pontos méretet, a rajztáblán található képkockák színét tárold dinamikus tömbben a struktúrán belül. Készíts függvényeket a **canvas** létrehozásához, valamint törléséhez.

### Rajzolás ( 6 pont )

Írd meg a **canvas\_print()** függvényt! A függvény kapjon egy **canvas**-t és rajzolja ki a rajztábla tartalmát a terminálra!

## Argumentumok I. ( 8 pont )

A program indításkor várjon 3 argumentumot, a rajztábla

- szélességét
- magasságát
- háttérszínét

Ezek alapján hozz létre egy `canvas`-t, majd írasd is ki. A kirajzolás után a program álljon le. Készülj fel arra is, hogy nem megfelelő számú argumentum érkezik, ilyenkor a program jelezze a hibát és álljon le.

*Tipp: Több futtatással is ellenőrizd, hogy a megfelelő alakú és színű rajztábla jelenik-e meg a kijelzőn különböző megadott argumentumokkal.*

*Megjegyzés: 3 argumentum esetén feltételezzük, hogy a megfelelő értékek érkeznek, a megadott sorrendben.*

## Szövegből enum konvertáló( 4 pont )

A háttérszín karakterlánc formában kapja a program. Készíts egy `color_converter()` függvényt, amely a kapott karakterlánc alapján visszaadja a színnek megfelelő elemet a felsorolási típusból.

## Téglalap ( 10 pont )

A kirajzolandó mintáinkat téglalapokból fogjuk létrehozni. Készítsd el a `rectangle` struktúrát, ami tartalmazza a bal felső, valamint jobb alsó sarkok koordinátáit, és a téglalap színét.

Írd meg a `canvas_fill()` függvényt! A függvény paraméterként várjon egy `canvas`-t és egy `rectangle`-t, majd rajzolja rá a `canvas`-ra a téglalapot. Azaz a rajztábla megfelelő pixeleinek a színét módosítsa a téglalap színére.

## Fájl olvasás ( 12 pont )

Hozd létre a `canvas_load_from_file()` függvényt! A függvény egy fájlnev alapján nyissa meg az adott fájlt és hozzon létre és rajzoljon meg egy `canvas`-t, a fájlban található információk alapján.

A fájl felépítése:

SZÉLESSÉG MAGASSÁG SZÍN

X1 Y1 X2 Y2 SZÍN

X3 Y3 X4 Y4 SZÍN

...

Az első sor a rajztábla méretét és a háttér alapszínét adja meg, míg a többi sor a `canvas`-ra rajzolandó téglalapokat (bal felső sarok koordinátái, jobb alsó sarok koordinátái, színe).

## Argumentumok II. ( 4 pont )

Amennyibe a programnak csak egy argumentumot adunk meg futtatáskor, feltételezze, hogy az egy fájlnev, hozza létre a fájl alapján a `canvas`-t, rajzolja ki, majd álljon le. A mellékelt **demo.txt** fájlt használhatod a program tesztelésére.